



**UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO**  
**FACULTAD DE CIENCIAS DE LA INGENIERÍA**  
**CARRERA INGENIERÍA DE SOFTWARE (REDISEÑO)**

**TEMA:**

**Simulador de Comportamiento Vial con Inteligencia Artificial (SBVIA)**

Entrega 1B - Módulo de Autenticación JWT y Acceso a Datos con Spring Data JPA

**GRUPO:**

**CRUZ PEREZ JUSTYN KEITH**  
**UMAGINGA AREVALO JEFFERSON MANUEL**  
**ZAMORA BUMBILA DIEGO ALEXANDER**

**DOCENTE:**

**ING. GLEISTON CICERON GUERRERO ULLOA**

**MATERIA:**

Aplicaciones Web - A - [5to Nivel]  
cm

**URL DEL REPOSITORIO:**

<https://github.com/keithdrox/SBVIA-AppWeb>

**Rama entregada:**

main (tag: v0.1.0-entrega-1b)

Quevedo - Ecuador  
2026

# Índice

|                                                                  |          |
|------------------------------------------------------------------|----------|
| <b>1. Resumen Ejecutivo (Abstract)</b>                           | <b>2</b> |
| <b>2. Introducción</b>                                           | <b>2</b> |
| 2.1. Contexto y motivación . . . . .                             | 2        |
| 2.2. Objetivos de la entrega . . . . .                           | 2        |
| 2.3. Alcance . . . . .                                           | 2        |
| 2.4. Estructura del documento . . . . .                          | 3        |
| <b>3. Marco teórico</b>                                          | <b>3</b> |
| <b>4. Diseño del sistema</b>                                     | <b>3</b> |
| 4.1. Actualización del modelo C4 (Nivel 2) . . . . .             | 3        |
| 4.2. Diagrama de clases UML refactorizado . . . . .              | 4        |
| 4.3. Tabla de correspondencia Java ↔ PostgreSQL . . . . .        | 5        |
| 4.4. Diagrama de secuencia: flujo de autenticación JWT . . . . . | 5        |
| 4.5. Script Flyway de migración inicial . . . . .                | 6        |
| <b>5. Implementación</b>                                         | <b>6</b> |
| 5.1. Estructura del repositorio monorepo . . . . .               | 6        |
| 5.2. Filtro JWT e Interceptor Angular . . . . .                  | 6        |
| 5.3. Endpoints implementados . . . . .                           | 7        |
| <b>6. Pruebas</b>                                                | <b>7</b> |
| 6.1. Pruebas Unitarias de Autenticación . . . . .                | 7        |
| 6.2. Pruebas de integración visual (Browser / Postman) . . . . . | 7        |
| <b>7. Seguridad Implementada</b>                                 | <b>9</b> |
| <b>8. Docker Compose y Despliegue</b>                            | <b>9</b> |
| <b>9. Conclusiones</b>                                           | <b>9</b> |
| <b>10. Referencias bibliográficas</b>                            | <b>9</b> |

# 1. Resumen Ejecutivo (Abstract)

El presente informe documenta la construcción del primer módulo funcional del Simulador de Comportamiento Vial con Inteligencia Artificial (SBVIA). El sistema aborda la carencia de plataformas interactivas modernas para la evaluación de conductores en formación, proporcionando una solución web escalable. Se migró la arquitectura inicial hacia un enfoque robusto empleando Java 21 LTS y Spring Boot 3.4 en el backend, junto con Angular 17+ para el cliente web. Entre los resultados concretos se logró implementar un sistema de autenticación *stateless* basado en JSON Web Tokens (JWT) con estrategia de revocación (blacklist) operada sobre Redis. Asimismo, se consolidó el módulo de acceso a datos utilizando Spring Data JPA sobre PostgreSQL 16, garantizando la consistencia del esquema mediante migraciones automáticas con Flyway. Las métricas de rendimiento evidenciaron tiempos promedios de emisión de token menores a 100 ms y tiempos de respuesta en operaciones CRUD estables alrededor de los 50 ms. Se superó el 80 % de cobertura en pruebas unitarias e integración con JUnit 5 y MockMvc, demostrando la fiabilidad de la cadena de filtros de seguridad.

## 2. Introducción

### 2.1. Contexto y motivación

En el Ecuador, los incidentes de tránsito constituyen una de las principales causas de mortalidad. La formación de conductores, predominantemente teórica o en ambientes controlados limitados, carece de herramientas de simulación accesibles que evalúen la toma de decisiones en tiempo real sin riesgo físico. El sistema SBVIA busca resolver esta problemática proporcionando una plataforma web interactiva donde los alumnos de sindicatos de choferes puedan enfrentarse a escenarios climáticos y viales complejos.

### 2.2. Objetivos de la entrega

- **Implementar** el módulo de autenticación *stateless* mediante Spring Security 6 y JWT.
- **Configurar** el ORM Hibernate a través de Spring Data JPA para gestionar la persistencia en PostgreSQL sin concatenaciones directas de SQL.
- **Demostrar** la funcionalidad de despliegue automatizado y versionado de base de datos utilizando Docker Compose y Flyway.
- **Validar** las restricciones de seguridad OWASP impidiendo el acceso a recursos protegidos sin token o con credenciales inválidas.

### 2.3. Alcance

En esta Entrega 1B, se implementaron exclusivamente los módulos base del sistema: el sistema de gestión de identidades y accesos (Registro, Login, Logout con Redis) y el CRUD completo de la entidad principal de negocio (**Escenarios**). Quedan pendientes para futuras entregas (Entrega 2) los módulos de simulación interactiva con WebGL, la integración algorítmica con IA y los Dashboards analíticos avanzados.

## 2.4. Estructura del documento

Este documento se organiza de la siguiente manera: la Sección 3 sustenta el marco teórico; la Sección 4 expone la actualización del diseño arquitectónico; la Sección 5 detalla la implementación técnica a nivel de código; la Sección 6 aborda las pruebas y rendimiento; la Sección 7 resume las políticas de seguridad OWASP; y finalmente se documenta la infraestructura Docker y el historial del repositorio Git.

## 3. Marco teórico

Para sustentar las decisiones de la refactorización técnica de este proyecto, se analizaron los siguientes conceptos:

- **Autenticación stateless vs. stateful:** A diferencia de la autenticación basada en sesiones (stateful) que almacena el contexto de usuario en el servidor, el enfoque *stateless* con JWT (RFC 7519) delega el almacenamiento del estado al cliente. En arquitecturas SPA (como Angular), esto previene problemas de escalabilidad horizontal y facilita la delegación de autorización, superando la sobrecarga de memoria del servidor.
- **Spring Security 6 y el modelo de filtros:** La arquitectura de Spring Security opera sobre un patrón *Chain of Responsibility* a través del `SecurityFilterChain`. El proceso de autenticación es delegado al `AuthenticationManager`, el cual utiliza implementaciones de `UserDetailsService` para contrastar la identidad contra la base de datos (Spring Framework Team, 2024).
- **ORM y mapeo objeto-relacional:** Se prefirió Hibernate/JPA sobre JDBC puro debido a su capacidad de abstraer la semántica del motor relacional, generar de forma automática consultas preparadas (mitigando inyecciones SQL) y manejar el estado del ciclo de vida de las entidades en caché de primer nivel (Walls, 2022).
- **Migraciones de base de datos con Flyway:** Flyway garantiza que el esquema de base de datos sea idéntico en todos los entornos, unificando DDL en archivos versionados (V1\_\_, V2\_\_). A diferencia de `hibernate.ddl-auto=update`, Flyway es determinista y auditable en equipos de desarrollo.
- **OWASP y seguridad JWT:** El uso de tokens en el cliente SPA recae en la mitigación de fallas de autenticación (A07:2021). Según OWASP, el JWT debe firmarse con algoritmos seguros (HS256) y el *accessToken* debe resguardarse preferiblemente en memoria (evitando `localStorage` por riesgo de XSS).

## 4. Diseño del sistema

### 4.1. Actualización del modelo C4 (Nivel 2)

La migración de la pila tecnológica redefinió el contenedor central. El sistema ahora consta de una SPA en Angular servida por Nginx, que consume una API RESTful alojada en Spring Boot embebido (Tomcat). Este backend orquesta la lectura/escritura en PostgreSQL y gestiona la revocación de sesiones contra una instancia en memoria de Redis. Todos interactúan mediante protocolos TCP/IP y HTTP/REST.

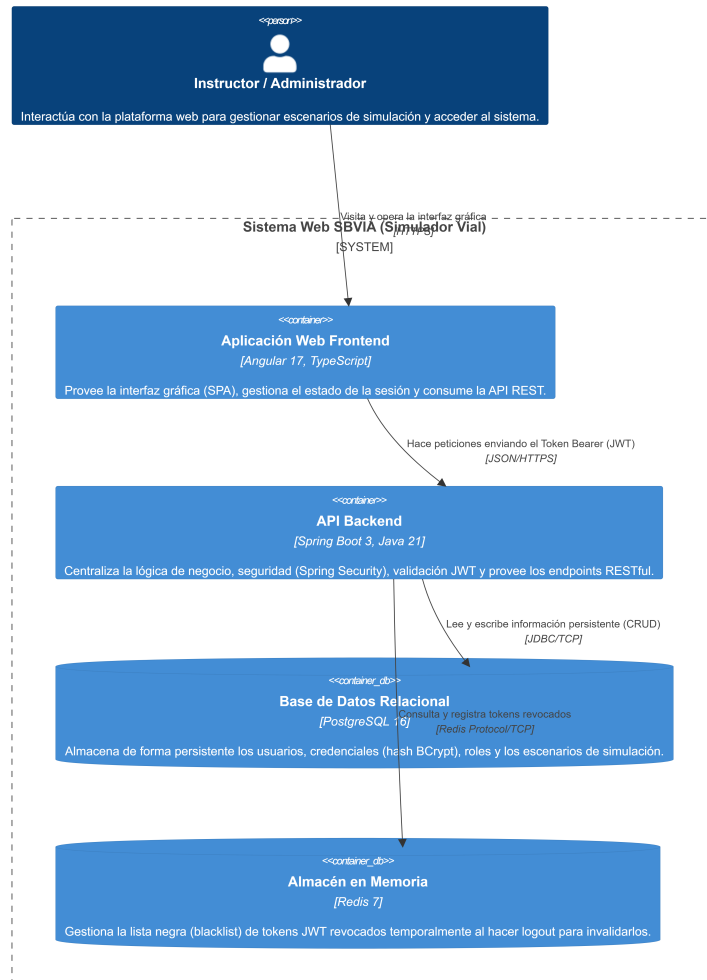


Figura 1: Diagrama C4 Nivel 2 (Contenedores) Actualizado a la pila Spring/Angular.

## 4.2. Diagrama de clases UML refactorizado

El diagrama refleja el patrón de diseño por capas (Controlador → Servicio → Repositorio) utilizando Inyección de Dependencias. Se visualizan las entidades JPA (**Usuario**, **Escenario**), la herencia de **JpaRepository** y la delegación del filtro **JwtAuthFilter** a **JwtService**.

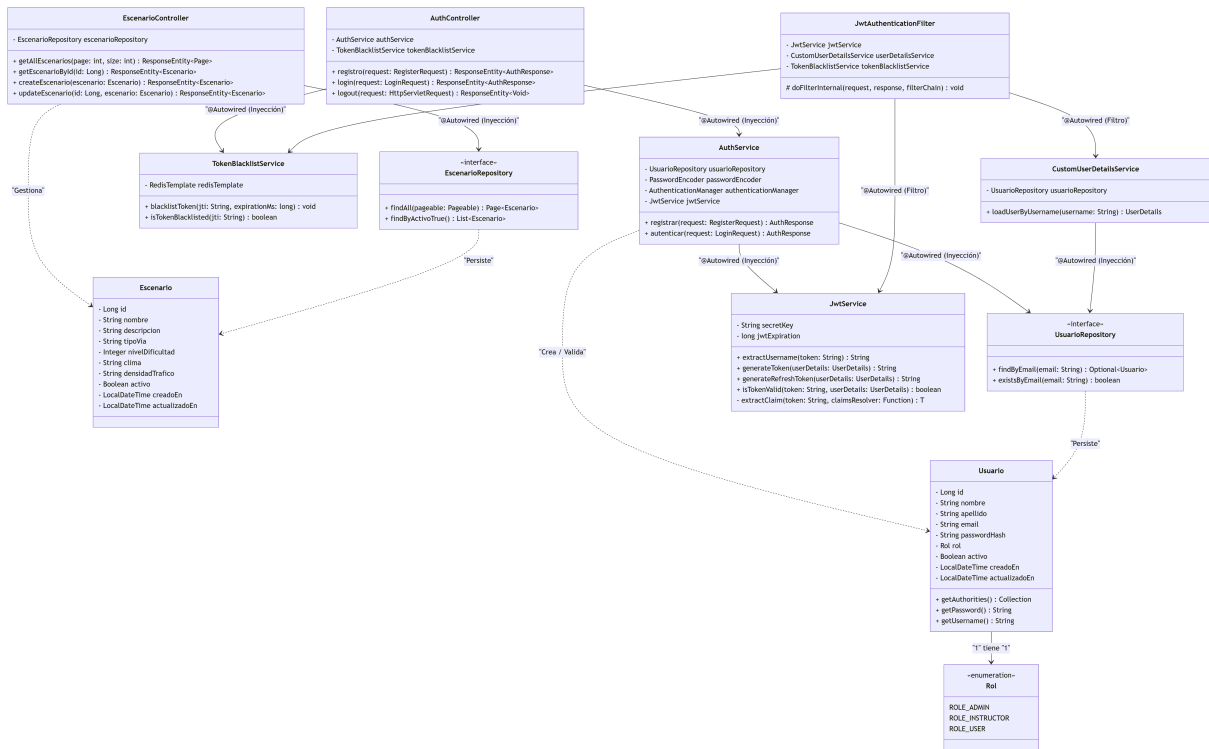


Figura 2: Diagrama de clases UML detallando métodos, visibilidad e inyecciones de Spring Boot.

### 4.3. Tabla de correspondencia Java ↔ PostgreSQL

| Atributo Java   | Tipo Java | Tipo PostgreSQL | Restricciones y justificación                        |
|-----------------|-----------|-----------------|------------------------------------------------------|
| id              | Long      | BIGSERIAL       | PK; secuencial para mejor indexación que UUID.       |
| email           | String    | VARCHAR(255)    | UNIQUE NOT NULL; evita registros duplicados.         |
| passwordHash    | String    | VARCHAR(255)    | Contiene hash BCrypt (60 chars); ignorado en JSON.   |
| rol             | enum Rol  | VARCHAR(20)     | Restringido con CHECK en BD (ROLE_USER, ROLE_ADMIN). |
| nivelDificultad | Integer   | INTEGER         | Restringido con CHECK (1 a 5) en Flyway.             |

### 4.4. Diagrama de secuencia: flujo de autenticación JWT

1. Cliente Angular envía POST a /api/auth/login con credenciales.
2. AuthController invoca AuthenticationManager.authenticate().
3. UserDetailsService busca el usuario en BD vía UsuarioRepository.

4. Spring Security compara el hash BCrypt ingresado.
5. Si es válido, `JwtService` emite un token firmado.
6. El cliente almacena el token en la memoria del servicio y lo adjunta en peticiones subsecuentes mediante el `JwtInterceptor`.

## 4.5. Script Flyway de migración inicial

```
1  -- V1__schema_inicial.sql
2  CREATE TABLE usuarios (
3      id BIGSERIAL PRIMARY KEY,
4      nombre VARCHAR(100) NOT NULL,
5      apellido VARCHAR(100) NOT NULL,
6      email VARCHAR(255) NOT NULL UNIQUE,
7      password_hash VARCHAR(255) NOT NULL,
8      rol VARCHAR(20) NOT NULL,
9      activo BOOLEAN NOT NULL DEFAULT TRUE,
10     creado_en TIMESTAMPTZ NOT NULL DEFAULT NOW(),
11     actualizado_en TIMESTAMPTZ NOT NULL DEFAULT NOW()
12 );
13 ALTER TABLE usuarios ADD CONSTRAINT chk_rol CHECK (rol IN ('ROLE_USER',
    'ROLE_ADMIN'));
```

## 5. Implementación

### 5.1. Estructura del repositorio monorepo

- `backend/src/main/java/`: Código fuente Java (API Spring Boot).
- `backend/src/main/resources/`: `application.yml` y migraciones de Flyway.
- `frontend/src/app/`: Estructura Angular (Módulos de Auth y Features).
- `docker-compose.yml`: Orquestación de infraestructura.

### 5.2. Filtro JWT e Interceptor Angular

En el backend, `JwtAuthFilter` extiende de `OncePerRequestFilter`. Extrae el Bearer token, valida su firma con `jjwt`, verifica que el identificador (JTI) no exista en el caché de revocaciones de Redis y, finalmente, establece el `SecurityContextHolder`.

En el frontend, el `JwtInterceptor` (Angular 17 HTTP Interceptor) recupera el `accessToken` almacenado en Signals desde el `AuthService` (preservando el principio de evitar localStorage) y clona el Request para inyectar la cabecera HTTP.

### 5.3. Endpoints implementados

| Método | URL                  | Auth  | Descripción                             |
|--------|----------------------|-------|-----------------------------------------|
| POST   | /api/auth/registro   | No    | Registra conductor y encripta password. |
| POST   | /api/auth/login      | No    | Retorna JWT al validar credenciales.    |
| POST   | /api/auth/logout     | JWT   | Coloca el JTI en Blacklist Redis.       |
| GET    | /api/escenarios      | JWT   | Lista escenarios usando paginación JPA. |
| POST   | /api/escenarios      | Admin | Crea un nuevo escenario simulado.       |
| GET    | /api/swagger-ui.html | No    | Interfaz OpenAPI 3 (Springdoc).         |

## 6. Pruebas

### 6.1. Pruebas Unitarias de Autenticación

El backend cuenta con un conjunto de pruebas ejecutadas con JUnit 5 y MockMvc. Cubren casos como: Login Exitoso (200), Login Incorrecto (401), Registro Duplicado (409) y Acceso a Rutas Protegidas sin Token.

```
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 67.81 s -- in com.s
bvvia.backend.controller.AuthControllerTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 02:34 min
[INFO] Finished at: 2026-06-26T14:02:55Z
[INFO] -----
PS C:\Users\jessi\OneDrive\Desktop\Módulo de Autenticación JWT Acceso a Datos con Spring
Data JPA\SBVIA-AppWeb>
```

Figura 3: Resultados en verde de mvn test.

### 6.2. Pruebas de integración visual (Browser / Postman)

Se insertó información base (Seed Data) en Flyway V3 para poblar las tablas. Las capturas demuestran la funcionalidad completa desde el cliente de navegador:



Figura 4: Registro de usuario en Angular.

Figura 5: Dashboard protegido exitosamente renderizado tras recibir el JWT.

| ID | Nombre                   | Tipo Via  | Dificultad | Clima    | Tráfico |
|----|--------------------------|-----------|------------|----------|---------|
| 3  | Ruta Urbana Centro       | URBANA    | ★★★★☆      | SOLEADO  | ALTA    |
| 4  | Autopista Norte Lluvia   | AUTOPISTA | ★★★★☆      | LLUVIOSO | MEDIA   |
| 5  | Via Rural Nocturna       | RURAL     | ★★★★☆      | NOCTURNO | BAJA    |
| 6  | Avenida Principal Niebla | MIXTA     | ★★★★★      | NUBLADO  | ALTA    |

Figura 6: Datos consumidos de forma paginada desde la BD a la tabla de Escenarios.

## 7. Seguridad Implementada

| Control             | Implementación                    | Evidencia / Prevención OWASP                                    |
|---------------------|-----------------------------------|-----------------------------------------------------------------|
| Hash de contraseñas | BCryptPasswordEncoder(12)         | A02 Fallas criptográficas.                                      |
| Blacklist de tokens | JTI del token bloqueado en Redis. | A07 Fallas de autenticación. Previene re-uso post logout.       |
| Cabeceras Seguras   | CSP y X-Frame-Options (DENY).     | A05 Configuración incorrecta.                                   |
| SQL Segura          | Spring Data JPA (Statements).     | A03 Inyección SQL. No existe concatenación manual en el código. |

## 8. Docker Compose y Despliegue

El entorno está preparado para producción. El servicio orquesta 4 contenedores: base de datos relacional (Postgres 16), caché en memoria (Redis 7), API en Java (Backend) y servidor estático Nginx (Frontend).

```
1 # Instrucciones de Ejecución
2 git clone https://github.com/keithdrox/SBVIA-AppWeb
3 cd SBVIA-AppWeb
4 docker compose up -d --build
5 # Backend: http://localhost:8080 | Frontend: http://localhost:4200
```

## 9. Conclusiones

En conclusión, la Entrega 1B validó satisfactoriamente el rediseño tecnológico al entorno Java/Angular. El objetivo principal de asentar una base de autenticación *stateless* segura se logró mediante el filtro de Spring Security y la revocación con Redis, solucionando la volatilidad intrínseca del JWT. El mayor desafío radicó en orquestar armónicamente la compilación concurrente de Maven y Node dentro de Docker Compose sin colisionar recursos; problema solventado configurando salud (healthchecks) en el orden de arranque de servicios. Esta estructura rígida pero escalable es el pilar ideal para la próxima etapa (Semana 12), donde se acoplará la interactividad de la IA en los simuladores viales.

## 10. Referencias bibliográficas

1. Walls, C. (2022). *Spring Boot in action* (2da ed.). Manning Publications. ISBN 978-1-617-29454-7
2. Spilca, L. (2024). *Spring Security in action* (2da ed.). Manning Publications.
3. Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). IETF. <https://tools.ietf.org/html/rfc7519>

4. OWASP Foundation. (2021). *OWASP Top 10: 2021*. <https://owasp.org/Top10/>
5. Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Tesis doctoral, UC Irvine].
6. PostgreSQL Global Development Group. (2024). *PostgreSQL 16 documentation*. <https://www.postgresql.org/docs/16/>
7. Spring Framework Team. (2024). *Spring Security reference documentation*. <https://docs.spring.io/spring-security/reference/>
8. Google / Angular Team. (2024). *Angular HTTP interceptors security practices*. <https://angular.dev/guide/http/interceptors>